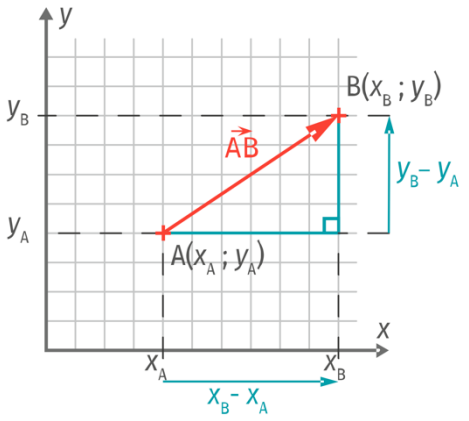
Contrôle Continu Programmation C++ - Année 2022/2023

Durée : 1h30 - Documents papiers autorisés

Exercice 1 : « Un peu de géométrie »

Dans un repère plan, un vecteur v(x,y) est un objet mathématique représentant un segment orienté. Les coordonnées x et y correspondent à la distance parcourue horizontalement (x) et verticalement (y) pour aller d'une extrémité à l'autre du segment. Dans l'exemple de la figure à droite le vecteur $\overrightarrow{AB}$ est construit à partir des points A(4,4) et B(10,8) et se note $\overrightarrow{AB}(6,4)$.



Q1 On suppose que la classe Point existe (avec getters et setters), écrire le code minimal de la classe Vecteur permettant d'exécuter le code suivant :

```
Point A(4,4);        // Création du Point A
Point B(10,8);       // Création du Point B
Vecteur AB1(6,4);    // Création du vecteur AB1 à partir de x=6 et y=4
Vecteur AB2(A,B);    // Création du vecteur AB2 à partir des points A et B
```

Q2 On se propose de créer le code nécessaire pour transformer un vecteur quelconque en vecteur unitaire (vecteur de norme unitaire). Pour rappel, la norme d'un vecteur $\vec{v}(x,y)$ est égal à $\sqrt{x^2+y^2}$.

Ecrire le code d'une méthode ET le code d'une fonction helper (et de toutes celles nécessaires si besoin) réalisant cette opération et permettant d'exécuter le code suivant :

```
AB1.unitaire();      // Appel de la méthode
unitaire(AB2);       // Appel de la fonction helper
```

Q3 On se propose de rajouter une méthode ou une fonction helper (justifier le choix) permettant de calculer le produit scalaire de 2 vecteurs. Écrire le code de cette fonction (et de toutes celles nécessaires si besoin). Donner un exemple d'utilisation de la fonction.

Q4 Pour des raisons de lisibilité de code, on se propose de remplacer la méthode/fonction de calcul du produit scalaire par une écriture mathématique plus simple à comprendre telle que v1*v2. Pour cela, utiliser la surcharge de l'opérateur *. Quelles sont les différentes manières d'écrire cette surcharge ? S'il y a plusieurs possibilités, choisissez-en une et écrire le code permettant d'exécuter :

```
Vecteur v1(4,3);
Vecteur v2(2,5);
double produit_scalaire = v1*v2 ;
```

Q5 On sait que deux vecteurs v1 et v2 sont perpendiculaires si leur produit scalaire v1*v2 = 0. On sait de même que deux vecteurs sont colinéaires s'il existe un nombre k tel que v2 = kv1. Ecrire le code des fonctions permettant de tester si deux vecteurs sont perpendiculaires ou colinéaires

Exercice 2 : « Un peu de statistiques »

On se propose d'effectuer quelques opérations statistiques sur des nombres stockés dans des tableaux. Pour cela, on utilisera une classe Data comprenant un tableau de réels.

Q1 Ecrire le code minimal de la classe permettant de créer un objet de type Data contenant des données réelles aléatoires, le nombre de données n à générer sera passé en paramètre du constructeur (par exemple, n=10000). Pour générer les données aléatoires, on utilisera la fonction rand() qui retourne un nombre entier aléatoire positif. La partie entière sera générée entre -100 et 100 et la partie décimale sera générée entre 0 et 99. La partie entière et la partie décimale seront ensuite assemblées pour constituer le nombre réel à stocker dans le tableau.

Q2 Ecrire le code de la méthode permettant de calculer la moyenne $\bar{x}$ des données stockées dans le tableau.

Q3 Ecrire le code de méthode permettant de calculer l'écart type σ des données stockées dans le tableau. Pour rappel, l'écart type d'une série de n valeurs est défini comme $\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$ avec $\bar{x}$ la moyenne.

Q4 On souhaite utiliser les fonctions moyenne et écart type pour modifier un objet de type Data en enlevant les données aberrantes, c'est-à-dire celles qui s'éloignent de plus de X fois l'écart-type. Ecrire le code de la méthode permettant de supprimer les données aberrantes, le X étant passé en paramètre de la fonction.

Q5 Ecrire cette même méthode en utilisant un des algorithmes de la STL. Pour cela, vous utiliserez la fonction std::remove_if dont les 3 paramètres sont l'itérateur de début, l'itérateur de fin et une expression lambda qui renvoie True si l'élément doit être supprimé. Après avoir supprimé toutes les valeurs répondants à l'expression lambda, l'algorithme remove_if renvoie l'itérateur de la dernière donnée valide.

Vous pouvez vous inspirer de l'exemple d'utilisation présenté ci-dessous qui supprime toutes les valeurs égales à 20 dans le tableau.

```
    // Initial size = 8
    std ::vector<int> vec1 = {10, 20, 30, 30, 20, 10, 10, 20};
    // The value to delete
    int to_delete = 20;
    // Delete all the values equal to 20
    // And returns an iterator on the last valid element (here 5th position)
    auto it = std ::remove_if(vec1.begin(), vec1.end(), [to_delete] (int x) {return
x ==  to_delete;});
    // Delete the last 3 values of the vector to get only the valid values
    // Vector is now {10, 30, 30, 10, 10}
    vec1.erase(it, vec1.end());
```